

SYSTEM DESIGN INTERVIEW

Design an E-commerce System

Microservices, Inventory Management & Order Orchestration

~ 35 minute read

© Study Guide — For personal use only

What's Inside

- 01** · Problem Overview
- 02** · Clarifying Questions
- 03** · Capacity Estimation
- 04** · System Architecture
- 05** · Core Design Decisions
- 06** · Data Models
- 07** · Order Placement Pipeline
- 08** · Inventory & Concurrency
- 09** · Search Architecture
- 10** · Recommendations

11 · Failure Modes

12 · Design Trade-offs

13 · Interview Playbook

01

Problem Overview

Building Amazon-scale e-commerce

Designing an e-commerce platform requires balancing availability, consistency, and scale. Amazon handles **~500M products**, **~1B registered users** (~300M MAU), and processes on the order of **1.6M orders/day** on an average day, with peaks of **10–13M+ orders/day** during Prime Day and Cyber Week.

The Scale Problem

- **Users:** 1 billion registered; ~300M monthly active
- **Catalog:** 500M products across all categories (electronics, clothing, books, groceries, ...)
- **Average traffic:** ~1.6M orders/day → **~18 orders/sec** sustained
- **Peak traffic:** ~13M orders/day on Prime Day → **~150 orders/sec**, with intra-day spikes of **500–1,000 ops/sec** in the first hour of a major sale
- **Search QPS:** ~500K queries/sec globally
- **Read-heavy:** 99% reads (browse, search, reviews) vs. 1% writes (orders, updates)

Key Challenges

- **Inventory consistency:** prevent overselling; real-time reservation conflicts across warehouses
- **Order atomicity:** payment → inventory deduction → fulfillment must look transactional across services
- **Search relevance:** 500M products; millisecond response; personalized by user history
- **Global distribution:** regional warehouses, VAT compliance, currency conversion
- **Recommendations:** personalized homepage; “customers also bought” at scale

02

Clarifying Questions

Scope and assumptions

Question	Answer / Assumption
Single product or multi-vendor marketplace?	Single vendor (Amazon.com); seller accounts optional (Phase 2)
Global or single-region?	Global (multi-region: US, EU, APAC); local currency & tax
Inventory real-time or eventual consistency ok?	Real-time for high-velocity SKUs; eventual ok for slow movers
Search: full-text only or faceted?	Full-text + facets (price, rating, seller, availability)
Return / cancellation handling?	Out of scope; assume successful orders only
Payment processing (in-house or 3rd party)?	3rd party (Stripe, PayPal); async webhook confirmation
Seller onboarding (FBA, FBM)?	FBA only (Fulfillment by Amazon); logistics out of scope

03

Capacity Estimation

Numbers to anchor your design

Product Catalog

- 500M products $\times$ 2 KB average metadata = **~1 TB** in product catalog DB
- Storage for images: 5 images/product $\times$ 500 KB average = **1.25 PB** in S3, served via CDN
- Indexing overhead: Elasticsearch inverted index ~30% of raw data = **~300 GB** across 50 shards

Order Volume

- Average: **~1.6M orders/day** $\rightarrow$ **~18 orders/sec**
- Peak (Prime Day / Cyber Monday): **~13M orders/day** $\rightarrow$ **~150 orders/sec**; flash-sale opening spikes hit **500–1,000 ops/sec**
- Order corpus: $1.6M \times 365 \times 10 \text{ yr} \approx$ **5.84B orders**
- Order record size: ~5 KB (header + items + addresses + status history) $\rightarrow$ **~29 TB hot** in PostgreSQL
- Fulfillment queue: Kafka topic; ~1.6M events/day average, ~13M/day peak; 7-day retention $\approx$ **0.4–0.6 TB**

Search QPS

- **500K queries/sec** globally
- Elasticsearch latency target: **<100 ms p99**
- Shards needed: $500K \text{ QPS} \div 1,000 \text{ QPS/shard} =$ **500 primary shards**; replicated $3\times =$ **1,500 shard copies**
- Replication factor: **RF = 3** for high availability

Bandwidth & Storage

- Product images: $5 \times 500 \text{ KB} \times 500M =$ **1.25 PB**; most served from CloudFront CDN
- Product detail pages: $300M \text{ MAU} \times 10 \text{ page views/month} \div 30 \text{ days} \approx$ **100M req/day** $\approx$ **1,157 req/sec**
- Image bandwidth: $\sim 1,000 \text{ req/sec} \times 500 \text{ KB} \approx$ **500 GB/day** = **50 MB/sec** sustained

KEY

Key Numbers

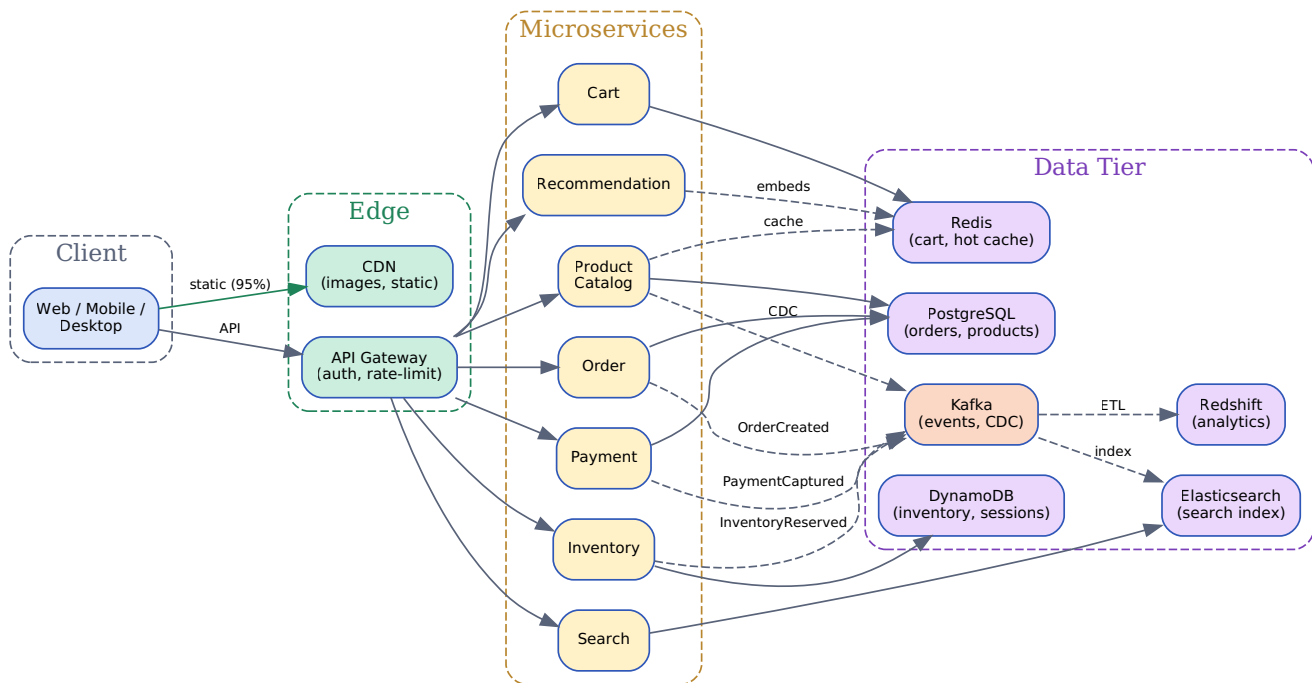
Orders: **~18 ops/sec avg**, **~150 ops/sec peak** · Order corpus: **5.84B rows** / **~29 TB hot** · Search QPS: **500K/sec** · ES shards: **500 $\times$ RF=3** · Catalog: **~1 TB + 1.25 PB** images

04

System Architecture

Microservices layered design

The platform is split into a **client tier**, a **service tier** of stateless microservices, and a **data tier** sized for each access pattern. The CDN absorbs image and static page traffic; the API gateway terminates auth and rate limits; each service owns its own datastore.



Client → CDN → API Gateway fans out to stateless microservices; each service owns its datastore. Dashed edges are async (Kafka).

Three-Tier Model

- **Client layer:** web browser, mobile app, desktop; HTTPS to load balancer
- **Service layer:** stateless microservices (Catalog, Search, Inventory, Order, Payment, Cart, Recommendations, Notifications)
- **Data layer:** PostgreSQL (orders/products), DynamoDB (inventory), Redis (cache/cart), Elasticsearch (search), S3 (images), Kafka (events), Redshift (analytics)

Load Balancer & API Gateway

- **Load balancer:** distributes traffic across API gateway instances (e.g., AWS ALB); 5-second health checks
- **API gateway:** single entry point; rate limiting, request validation, routing to microservices

- **Service discovery:** dynamic registry (Consul, Eureka, or Kubernetes service mesh); resolves DNS to healthy instances

05

Core Design Decisions

Why this architecture

Microservices Decomposition

- **Product Catalog:** read-heavy; immutable metadata; caches product details in Redis
- **Search:** Elasticsearch backend; full-text search, faceting, autocomplete
- **Inventory:** single source of truth for stock; enforces invariants via optimistic locking
- **Cart:** user's temporary shopping cart; stored in Redis (session-like); TTL = 30 days
- **Order:** creates orders; coordinates Inventory & Payment; publishes events to Kafka
- **Payment:** PCI-compliant; delegates to Stripe/PayPal; handles retries & idempotency
- **Recommendation:** offline batch + online serving; collaborative filtering + content-based
- **Notification:** sends order confirmation, shipment updates, promotional emails

Eventual Consistency in Inventory

- **Approach:** optimistic locking with version numbers; on conflict, client retries with the fresh inventory state
- **Conflict resolution:** "Sorry, only 2 left" message; offer similar items from Recommendations
- **Saga pattern:** if payment succeeds but inventory reserve fails, compensation marks payment for refund
- **Reservation TTL:** inventory hold expires after 15 min if order isn't confirmed; stock returns to the pool

Asynchronous Event-Driven Pipeline

- **Kafka broker:** central event bus; services publish domain events (OrderCreated, PaymentConfirmed, InventoryReserved)
- **Consumers:** Notification, Fulfillment, Analytics subscribe independently
- **Benefits:** loose coupling; if Notification is down, the order is still created and Kafka retains the event for replay
- **Retention:** 7 days; allows late subscribers to catch up

Kafka Partitioning

- **Topic orders.created:** peak ~150 ops/sec sustained, 1K ops/sec in flash bursts; partition by user_id for ordered per-user history
- **Partition count:** ~64 (~15 ops/sec/partition at peak; 4× headroom for hot users) with RF = 3
- **Topic inventory.reserved:** partition by sku; ~32 partitions to colocate writes for hot SKUs without excessive fan-out

- **Topic catalog.cdc:** partition by product_id for index ordering into Elasticsearch

Key Tables (PostgreSQL)

```
-- Product Catalog
CREATE TABLE products (
  product_id    BIGINT PRIMARY KEY,
  title         VARCHAR(255),
  description    TEXT,
  price         DECIMAL(10,2),
  category_id   INT,
  seller_id     BIGINT,
  rating        FLOAT,
  reviews_count INT,
  created_at    TIMESTAMP,
  updated_at    TIMESTAMP,
  INDEX idx_category (category_id)
);

-- Inventory (per-warehouse stock; optimistic lock)
CREATE TABLE inventory (
  sku           VARCHAR(50) PRIMARY KEY,
  product_id    BIGINT NOT NULL,
  quantity      INT,
  reserved      INT DEFAULT 0,
  version       INT DEFAULT 0,
  warehouse_id  INT,
  updated_at    TIMESTAMP,
  UNIQUE (product_id, warehouse_id),
  INDEX idx_product (product_id)
);

-- Orders (sharded by user_id)
CREATE TABLE orders (
  order_id      BIGINT PRIMARY KEY,
  user_id       BIGINT NOT NULL,
  order_date    TIMESTAMP,
  total_amount  DECIMAL(10,2),
  status        ENUM('created', 'payment_pending',
                    'confirmed', 'shipped', 'delivered'),
  shipping_address TEXT,
  INDEX idx_user (user_id),
  INDEX idx_status (status)
);
```

```
-- Order line items
CREATE TABLE order_items (
  order_item_id BIGINT PRIMARY KEY,
  order_id      BIGINT NOT NULL,
  product_id    BIGINT,
  sku           VARCHAR(50),
  quantity      INT,
  price         DECIMAL(10,2),
  INDEX idx_order (order_id)
);
```

Redis Keys (Cache & Sessions)

```
# Cart (session-scoped)
cart:<user_id>      -> { product_id: quantity, ... } # TTL = 30d

# Session
session:<sid>       -> { user_id, logged_in_at }      # TTL = 24h

# Product detail cache
product:<id>        -> JSON(title, price, rating, ...) # TTL = 1h

# Inventory cache (read-through)
inventory:<sku>     -> { quantity, reserved, version } # TTL = 5m

# Idempotency (per checkout request)
idem:<merchant>:<k> -> { status, body_hash, response } # TTL = 24h
```

INSIGHT

Why DynamoDB for inventory?

Inventory writes hit ~150 ops/sec average and burst to 1K+ ops/sec on a flash sale, all on a tiny key space (the SKU). DynamoDB gives single-digit-ms single-row writes with conditional updates (the optimistic-lock primitive) without us having to shard a relational DB by SKU. Postgres still owns the money path (orders + payments) where ACID joins matter.

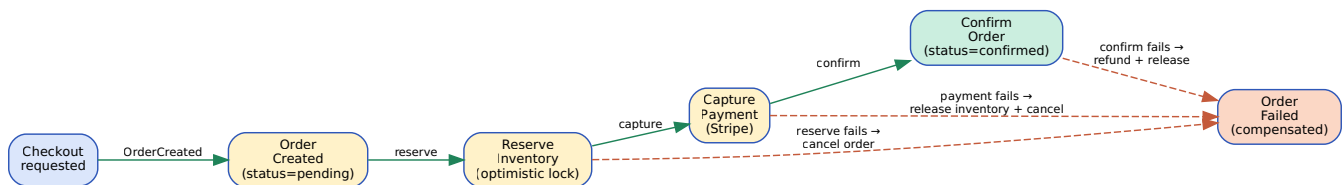
07

Order Placement Pipeline

Write path and saga pattern

Request Flow

1. User adds items to cart (Cart Service stores in Redis)
2. User clicks *Place Order*; Order Service receives the checkout request (with an idempotency key)
3. Order Service initiates payment via Payment Service (calls Stripe API)
4. Payment Service performs fraud check + card auth; returns success/failure
5. On success: Order Service calls Inventory Service to reserve stock (`sku`, `quantity`, `version`)
6. Inventory Service validates stock and updates with optimistic locking; returns `reservation_id`
7. Order Service creates the Order record (`status='confirmed'`); publishes `OrderCreated` to Kafka
8. Kafka triggers Fulfillment Service (async) → updates the warehouse system and starts picking
9. Kafka triggers Notification Service → sends the order-confirmation email
10. Client receives confirmation; order complete



Saga: each forward step has a compensation. Compensation arrows fire only on the failure of the next forward step.

Saga Pattern for Compensation

- **Scenario:** payment succeeds but inventory reserve fails (stock exhausted by a concurrent order)
- **Compensation:** Order Service marks the order failed; publishes `PaymentRefund`; Payment Service issues the refund
- **Idempotency:** every step is idempotent (same request twice = same result); refund is a no-op if already done
- **Timeouts:** payment timeout 30 s; inventory reserve timeout 10 s; on expiry, compensation fires

Reservation TTL

- **Add to cart:** no reservation yet; just a note in the Redis cart
- **Checkout begins:** Inventory Service reserves stock for 15 minutes
- **Order confirmed:** reservation converted to a permanent deduction (`quantity -= qty`)
- **Order fails / abandoned:** reservation expires after 15 min; stock returns to *available*

- **Benefit:** prevents overselling; still allows concurrent checkouts during the 15-minute window

TIP

Saga vs 2PC

Two-phase commit gives atomicity but blocks on the coordinator and dies at regional partitions — fatal at peak Prime-Day load. Sagas accept transient inconsistency windows (“payment captured but order not yet confirmed”) in exchange for availability. Idempotency on every step is the price of admission.

08

Inventory & Concurrency

Preventing overselling

Optimistic Locking Strategy

- **Version column:** each inventory row carries a version (starts at 0)
- **Reserve:** conditional update — succeeds only if the version matches
- **Conflict:** if the version doesn't match, the update fails and the client retries with fresh state
- **No locks:** many concurrent reservers race; one wins (increments version), the rest retry
- **Performance:** non-blocking; high throughput; only occasional retries on contention

```
-- Optimistic reserve. Returns 0 rows if someone beat us.
UPDATE inventory
  SET quantity = quantity - :qty,
      reserved = reserved + :qty,
      version  = version + 1,
      updated_at = NOW()
WHERE sku = :sku
  AND version = :expected_version
  AND quantity - reserved >= :qty;

-- DynamoDB equivalent (single round-trip):
--   ConditionExpression: 'version = :v AND quantity - reserved >= :qty'
--   UpdateExpression:   'SET quantity = quantity - :qty,
--                        reserved = reserved + :qty,
--                        version  = :v + 1'
```

Overselling Prevention

- **Invariant:** $\text{quantity} \geq \text{reserved}$ (available = quantity – reserved)
- **Reserve check:** $\text{available} \geq \text{order_qty}$ before attempting the update
- **Double-book handling:** on reserve failure, suggest alternates from the Recommendation Service
- **Flash-sale scenario:** 100 units, 1,000 concurrent checkouts → first 100 succeed, remainder get “out of stock”

Inventory Write Throughput

- **Average:** 1.6M orders/day × ~3 line items = ~5M reserves/day → ~55 ops/sec sustained
- **Peak:** 13M orders/day × 3 ≈ 39M reserves/day → ~450 ops/sec, with hot-SKU spikes of 1–5K ops/sec

- **DynamoDB sizing:** on-demand provisioning, hot-partition mitigation via SKU-suffix sharding for the top 1% of SKUs
- **Retry budget:** at 1% contention rate on hot SKUs, allow 3 retries before falling back to “out of stock”

Reservation TTL Mechanism

- **Background job (every minute):** scan reservations older than 15 minutes
- **Expiry logic:** `UPDATE inventory SET reserved = reserved - qty WHERE created_at < NOW() - INTERVAL 15 MIN`
- **Notification:** publish `ReservationExpired` for downstream consumers
- **Idempotency:** the job is idempotent; a second run finds no expired reservations and is a no-op

Indexing Strategy

- **Index structure:** `product_index` sharded by `hash(product_id) mod 50`; replica factor 3
- **Inverted index:** term → document IDs; e.g., `samsung` → [`prod_123`, `prod_456`, ...]
- **Vector search:** product embeddings as a `dense_vector` field; supports semantic queries (e.g., “cozy reading light”)
- **Refresh interval:** 30 s (near real-time); new products searchable within ~1 minute

Dual-Write to ES via CDC

- **Source of truth:** PostgreSQL `products` table
- **Pipeline:** Debezium / logical decoding → `Kafka catalog.cdc` → ES indexer consumer
- **Why CDC, not synchronous dual-write:** synchronous writes to PG + ES double the write latency and create partial-failure modes (in DB but not in index, or vice versa). CDC keeps the write path on the DB only and lets the index catch up asynchronously.
- **Lag budget:** 30–60 s end-to-end (DB commit → ES searchable); acceptable for catalog updates
- **Replay:** on indexer failure, replay from the last committed Kafka offset; the index is rebuildable from the CDC stream

Query Features

- **Full-text:** Boolean AND/OR, phrase queries, wildcards, fuzzy match (typo tolerance)
- **Faceting:** aggregations on category, price range, rating, seller; powers the filter UI
- **Autocomplete:** prefix query on title; top 10 suggestions in <50 ms
- **Ranking:** relevance score = TF-IDF + custom signals (rating, `reviews_count`, `sales_velocity`)

Read Path with Caching

- **Search latency target:** <100 ms p99 for ES + result assembly
- **Cache layer:** popular searches in Redis (e.g., `iphone 15` → cached results, TTL 1 h)
- **Product detail:** on result click, fetch product from Redis (HIT expected); on MISS, query Postgres + refill cache
- **Hit rates:** ~80% on searches (long-tail not repeated); ~95% on product detail (hot products checked repeatedly)

10

Recommendations

Personalization at scale

Two-Stage Approach

- **Offline (daily batch):** train a collaborative-filtering model on user-product interactions; generate top-100 recommendations per user
- **Online (request time):** fetch cached recommendations; apply real-time filters (already-purchased, out-of-stock); return top-20

Offline Batch Job

- **Input:** user-product interaction matrix (views, purchases, ratings) from Cassandra or Kafka events
- **Algorithm:** Alternating Least Squares (ALS) matrix factorization or deep neural network embeddings (Word2Vec-style)
- **Output:** embedding vectors per user / per product; stored in Redis or Cassandra for <10 ms lookup
- **Latency:** nightly Spark job; 2–4 hours; results deployed by morning

Online Serving

- **Request:** user_id → Recommendation Service → fetch embeddings from cache → compute similarity to hot products
- **Filtering:** remove items in the user's purchase history; remove out-of-stock items
- **Personalization:** rank by predicted rating using extra features (price sensitivity, category preference)
- **A/B testing:** homepage variants on 10% of users; winner (higher CTR / revenue) rolled out to 100%

11

Failure Modes

Scenarios and recovery

Failure	Cause	Impact	Mitigation
Payment Service down	Stripe API timeout; network	Checkout blocked; users cannot place orders	Retry with exponential backoff; circuit-breaker after 10 failures; queue orders for async processing
Inventory Service partition	DB network split	Reads fail; product page incorrectly shows "out of stock"	Read from cache (stale ok); eventual consistency; alert ops to repair partition
Search (Elasticsearch) unavailable	Cluster failure; node crash	Search results unavailable; "try again later"; browse by category as fallback	Replica shard failover (RF=3); add nodes; reindex from CDC stream
Order Service crashes	Unhandled exception in checkout	In-flight orders lost; user gets 500; may retry with duplicate payment	Idempotent request keys (at-most-once); reconciliation job merges duplicates
Kafka broker down	Hardware failure; network	Events cannot be published; async pipelines stall; fulfillment delayed	Kafka replication (RF=3); partition leader election; manual failover to replica broker
Redis cache crash	Memory exhaustion; hardware	Cart data lost; users re-add items; sessions expire	Redis Sentinel replicas; AOF + RDB snapshots; warm cache from DB on restart

WARNING

Graceful Degradation

Prioritize the money path. Browse and search can degrade to cached / stale data; cart can fall back to its DB mirror; recommendations can serve a non-personalized fallback. Never block checkout on a non-critical dependency — a failed promo banner must not break the order pipeline.

12

Design Trade-offs

Decisions and rationale

Decision	Choice	Trade-off
Orders datastore	PostgreSQL (sharded by user_id, with read replicas)	Strong ACID for the money path and easy joins for fulfillment, but operationally heavier than DynamoDB; alternative NoSQL (DynamoDB / Cassandra) scales writes more cheaply but forces you to roll your own multi-row consistency for order + items + payment status.
Inventory consistency	Eventual within a region (optimistic locking + 15-min reservation TTL); strong only inside the reserve transaction	High throughput, no global locks, occasional retries on contention; alternative strong/global consistency would block hot SKUs during flash sales and cap throughput at a few hundred ops/sec/SKU.
Distributed transaction model	Saga (event-driven compensation via Kafka) across Order → Payment → Inventory	Loose coupling, high availability, each step independently scalable; trade-off is intermediate inconsistency windows ("payment captured but order not yet confirmed") and the requirement that every step be idempotent. Alternative 2PC gives atomicity but couples services, blocks on coordinator failure, and does not survive cross-region partitions.
Concurrency control on stock	Optimistic locking with version column	Lock-free, scales horizontally, retries are cheap; alternative pessimistic SELECT ... FOR UPDATE serializes hot SKUs and creates lock convoys at peak.
Search index population	Async index from CDC / Kafka into Elasticsearch (eventual, ~30–60 s lag)	New products are searchable within a minute, write path stays cheap; alternative synchronous dual-write to Postgres + ES doubles write latency and creates partial-failure modes (in DB but not in index).
Cart storage	Redis (TTL = 30 days), no DB write until checkout	Sub-millisecond reads/writes, cheap, naturally session-scoped; trade-off is that a Redis cluster failure loses in-flight carts (mitigated by AOF + replicas). Alternative DB-backed carts survive failures but inflate write load by ~10–100× and add latency to every "add to cart".
Recommendations freshness	Offline ALS batch (daily) + online filter	Cheap to train, deterministic, A/B-testable; trade-off is staleness for new users / new items. Alternative real-time online learning is more responsive but harder to validate and prone to feedback loops.

Decision	Choice	Trade-off
Read scaling	CDN + Redis cache for product detail (~95% hit)	Big origin offload, low latency; trade-off is cache invalidation complexity (price/stock changes must purge edge + Redis).
Sharding strategy	Postgres sharded by <code>user_id</code> for orders; ES sharded by <code>product_id</code> hash	Aligns each query with a single shard; trade-off is that cross-user analytics needs a separate OLAP path (warehouse / Redshift).

KEY

Headline tension

The whole architecture is a balancing act between (a) the **money path**, which wants strong consistency, durability, and audit, and (b) the **browsing path**, which wants global scale, low latency, and cheap reads. Microservice boundaries are drawn so each side can pick its own consistency model.

13

Interview Playbook

How to ace this design (45 minutes)

Opening (5 min): Clarifying Questions

- Confirm scope: single-vendor Amazon or multi-vendor marketplace? (single-vendor; seller model in Phase 2)
- Scale: **1B users, 500M products, ~1.6M orders/day average, ~13M/day peak**
- Consistency: real-time inventory or eventual ok? (real-time on hot SKUs; eventual on slow movers)
- Geography: single region or global? (global; VAT & local fulfillment complicate the design)

Capacity Estimation (5 min)

- **~1.6M orders/day** → **~18 ops/sec average; ~13M/day peak** → **~150 ops/sec** (flash bursts of 500–1,000 ops/sec)
- 500M products × 2 KB = ~1 TB catalog; 500K QPS search ⇒ ~500 ES shards × RF=3
- Bandwidth: 100M page views/day × 5 images × 500 KB = ~50 MB/sec sustained to CDN
- Order corpus: **5.84B rows × 5 KB ≈ 29 TB hot** in PostgreSQL

Architecture (10 min)

- Client: web / mobile / desktop → HTTPS to load balancer
- Services: Catalog, Search (ES), Inventory, Cart (Redis), Order, Payment, Recommendations, Notifications
- Data: PostgreSQL (orders, products, users); DynamoDB (inventory); Redis (cache, sessions, cart); ES (search); S3 (images); Kafka (events)
- Draw the diagram with dashed lines for async (Kafka) flows

Deep Dives (20 min): Pick 2–3

- **Inventory & concurrency:** optimistic locking, version columns, reservation TTL, overselling
- **Order pipeline & saga:** transactional flow, compensation on payment failure, idempotency keys
- **Search:** ES sharding, inverted index, faceting, autocomplete, CDC dual-write, caching
- **Recommendations:** offline ALS training, online embedding lookup, filtering, A/B testing
- **Failure recovery:** circuit breakers, fallbacks, replication, Kafka durability

Wrap-Up (5 min): Trade-offs

- Eventual vs strong consistency: chose eventual for inventory (reservation TTL) to avoid global locks at peak
- Real-time vs offline recommendations: offline batch is sufficient (daily retraining); online serving adds personalization
- Microservices vs monolith: independent scaling (Search and Order have completely different load shapes)
- End with: “the key insight is inventory management — preventing overselling at **~150 ops/sec peak** requires lock-free concurrency control”

KEY

Key Numbers to Memorize

1B users, 500M products · **~1.6M orders/day (~18 ops/sec avg); ~13M orders/day peak (~150 ops/sec)** · 500K QPS search; ES 500 shards × RF=3; <100 ms p99 · Inventory: optimistic locking, 15-min reservation TTL · Order corpus: **5.84B rows / ~29 TB** hot · Recs: offline ALS daily + online <10 ms lookup.

What Interviewers Are Testing

- **Inventory concurrency:** can you prevent overselling? do you understand optimistic locking?
- **Order atomicity:** how do you make payment + inventory deduction look transactional? what if one fails?
- **Scale intuition:** 500M products → how many shards? 500K QPS → how many ES nodes?
- **Trade-off reasoning:** why eventual consistency for inventory? why offline batch for recommendations?
- **Resilience:** what happens if Payment Service times out? if a Kafka broker crashes?

INSIGHT

Closing line

“The hardest part of e-commerce design is preventing overselling while sustaining ~150 ops/sec on hot SKUs at peak. Optimistic locking with version numbers + reservation TTLs solves this without global locks; the saga pattern then ties payment, inventory, and order together with compensation rather than 2PC.”